

Random Augmentation vs. Canonicalization for Laplacian Positional Encodings

`map` and `random_augmented` tie at matched compute on $k=16$; both beat `oap` at $h \geq 128$.
`ogbg-molpcba`, GIN, 3 seeds, partial completion.

`ogbg-molpcba` experiments, GIN backbone

May 4, 2026

Abstract

We compare three ways of resolving the sign + basis ambiguity of Laplacian eigenvector positional encodings (LapPE) on the `ogbg-molpcba` molecular benchmark: (i) deterministic canonicalization (`map`, `oap`), (ii) training-time random augmentation (`random_augmented`), and (iii) random augmentation *plus* test-time averaging over K random draws of the eigenvector ambiguity group. Five sweeps (108 runs): Sweeps 1–3 under a fixed 200-epoch / patience-15 protocol at $k \in \{3, 8, 16\}$; Sweep 4 lifts the budget to 500 ep / no early stopping at $k=16$ for `random_augmented` only; **Sweep 5 gives `map` and `oap` the same 500-ep budget at $k=16$ (matched compute).**

Honest bottom line, at matched compute on $k=16$ / GIN / `molpcba` / 3 seeds:

- `map` and `random_augmented` are *statistically tied* at every hidden dim tested (Welch $p \geq 0.24$ at every h).
- Both *do* significantly beat `oap` at $h=128$ ($p < 0.001$); `random_augmented` beats `oap` at $h=512$ ($p \approx 0.04$); at $h=16$ all three are tied.
- Earlier readings (e.g. “`random_augmented` dominates with sufficient compute” from Sweep 4 alone, or “`oap` wins everywhere” from Sweep 3 alone) were apples-to-oranges artefacts of unmatched compute — `map` also gains +0.0088 to +0.0144 AP under matched compute, indistinguishable in scale from `random_augmented`’s gain.

Scope: these conclusions apply to $k=16$ only. The matched-compute test was not run at $k \in \{3, 8\}$. The Sweep 1 $k=3$ / $h=512$ `oap`-edge is not re-tested. Spielman, MaxAbs, Abs, and the bare `none` baseline are out of scope. `ogbg-molpcba` only; GIN backbone only; $h \in \{16, 128, 512\}$ only; 3 seeds.

Contents

1	What problem are we solving?	2
2	Exact parameter settings	3
3	Results	4
3.1	Sweep 1: $k=3$, $K=8$ — random augmentation wins; one apparent <code>oap</code> edge (likely compute artefact)	4
3.2	Sweep 2: $k=8$, $K=16$ — random augmentation wins at every hidden dim	4
3.3	Sweep 3: $k=16$, $K=16$ — under fixed compute, <code>oap</code> appears to win at every hidden dim	4

3.4	Sweep 4: $k = 16$, 500 epochs, no early stopping — random augmentation wins everywhere	5
3.5	Sweep 5: $k = 16$, matched-compute follow-up — <code>map</code> catches up, <code>oap</code> does not	5
3.6	Side-by-side: at matched compute, no clear winner	6
3.7	Visual summary — saturation flips the $k=16$ verdict	6
4	Interpretation	6
4.1	Matched compute on $k=16$: <code>map</code> $\approx$ <code>random_augmented</code> $>$ <code>oap</code>	6
4.2	Why does training compute matter so much at large k ?	7
4.3	Test-time averaging matters less than longer training	8
4.4	Why the prior <code>oap</code> “wins” were not robust	9
5	Main conclusions	9

1 What problem are we solving?

Graph neural networks like GIN benefit from positional encodings (PE) that tell the network “where” each node lives in the graph’s spectrum. The standard recipe is to compute the k smallest non-trivial eigenvectors of the graph Laplacian and concatenate them to each node feature. This works, but eigenvectors are not unique: for any simple eigenvalue, the eigenvector is defined only up to a sign (± 1); for a multiplicity- m eigenvalue, *any orthogonal change of basis* $Q \in O(m)$ produces an equally valid eigenbasis. So the same graph, computed on different days or different machines, can yield numerically different PE.

Three ways to handle this ambiguity:

Canonicalization. Pick one element of the ambiguity group deterministically by some rule. Examples:

- `map` — maximum-absolute-position rule per column.
- `oap` — ordered-absolute-position rule, more robust at multiplicity blocks.

The model sees the *same* PE every epoch and at test time.

Random augmentation. (`random_augmented`.) Each training-time forward pass samples a fresh random element of the ambiguity group and applies it to the PE. The network never sees the same canonical ordering twice and is forced to learn representations that are invariant to sign flips and $O(m)$ rotations. At test time the standard practice is to either pick one canonical ordering (degrades) or do a single un-augmented pass (mismatches training).

Random augmentation + test-time averaging. (`random_augmented` with K -sample averaging at test time.) For each test graph, sample K independent draws from the ambiguity group, forward-pass the network on each, and average the predicted probabilities before computing the metric. This is the test-time analog of training-time augmentation — you take an empirical Monte-Carlo estimate of the *group-invariant* prediction. Inspired by the `test_with_randomized_invariance` helper in Siegel *et al.*, [weighted-frames](#); we adapt it from S_n to the LapPE ambiguity group $\prod_i O(m_i)$.

2 Exact parameter settings

Both sweeps share everything except k and the test-time sample budget K .

Knob	Value
Dataset	ogbg-molpcba (molecular property prediction)
Backbone	GIN, 5 layers
PE type	eigvec (Laplacian eigenvectors)
Eigval scaling	off
Hidden dim h	{16, 128, 512}
Number of eigenvectors k	3 (sweep 1) / 8 (sweep 2) / 16 (sweeps 3, 4, 5)
Cache eigenvectors	$\max(k, 15)$ (slice at runtime)
Batch size	32
Learning rate	1×10^{-3}
Optimizer / scheduler	Adam, default cosine
Epoch budget (cap)	200 (sweeps 1–3) / 500 (sweeps 4, 5)
Early stopping patience	15 (sweeps 1–3) / 999 , i.e. disabled (sweeps 4, 5)
Seeds	{0, 1, 2} (3 seeds per cell)
Canonicalizations	sweeps 1–3: all three; sweep 4: random_augmented only; sweep 5: map and oap only (match)
Test-time samples K	8 (sweep 1) / 16 (sweeps 2, 3, 4, 5)
Total runs	$27 + 27 + 27 + 9 + \mathbf{18} = 108$
Hardware	$2 \times$ NVIDIA A40 (48 GB), 6 workers per GPU
Software	Python 3.10, PyTorch 2.10 (CUDA 12.8), conda env schnet-angle ; OGB / PyG installed via

Table 1: Hyperparameters across the five sweeps. Sweeps 1–3 use a fixed 200-epoch / patience-15 protocol; sweep 4 is the $k=16$ saturation test for **random_augmented** (lifts the budget to 500 epochs / disabled early stopping); **sweep 5 is the matched-compute follow-up** that gives **map** and **oap** the same 500-epoch / disabled-early-stopping budget so the head-to-head is fair. $K=8$ at $k=3$ exhausts the sign group $\mathbb{Z}_2^3$; $K=16$ at $k=8$ samples 6.25% of $\mathbb{Z}_2^8=256$; $K=16$ at $k=16$ samples 0.024% of $\mathbb{Z}_2^{16}=65,536$.

Reproducibility. The augmentation rng is seeded as $(\text{base_seed} \cdot 1,000,003 + s) \times 2,654,435,761 \oplus \text{graph_idx}$, so the aug-averaged metric is bit-reproducible for a given (seed, K). The training loop seeding (`torch.manual_seed`, cuDNN determinism, `worker_init_fn`) is what `src/training/seed_everything` provides — not bit-reproducible across hardware/CUDA versions; see `scripts/train_molecular.py` for the exact call. Per-seed results may therefore differ slightly from a clean re-run on different hardware.

Asymmetric test-time protocol. For **map/oap**, the test loader returns the deterministic canonical ordering. For **random_augmented**, the test loader returns the eigenvectors with the *cached* sign pattern from the one-time **lappe_cache** build (essentially one arbitrary-but-fixed sign assignment per graph), *not* a fresh draw. This makes the single-pass **best_test_ap** comparable across methods in the sense of being deterministic, but the **random_augmented** pattern is not its training-distribution mode — which is why we also report **best_test_ap_aug16** (averaged over $K=16$ random draws from the ambiguity group at test time).

3 Results

3.1 Sweep 1: $k = 3$, $K = 8$ — random augmentation wins; one apparent oap edge (likely compute artefact)

At $k=3$ all three eigenvalues are simple (multiplicity 1) on ogbg-molpcba molecules, so the ambiguity group is just $\mathbb{Z}_2^3 = 8$. We picked $K=8$ to cover it exactly in expectation. **random_augmented** wins at $h=16$ and $h=128$; **oap** narrowly takes $h=512$. This single $k=3/h=512$ **oap** edge is the only cell across the four sweeps where **random_augmented** appears to lose under fixed compute — and Sweep 4 (Table 5) shows that the analogous Sweep 3 **oap** wins at $k=16$ disappear once **random_augmented** is given enough epochs to internalise invariance, suggesting the same mechanism here.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=8$)	Δ aug
map	16	5,744	0.0626 ± 0.0025	—	—
oap	16	5,744	0.0641 ± 0.0051	—	—
random_augmented	16	5,744	0.0662 ± 0.0015	0.0663 ± 0.0015	+0.0001
map	128	202,752	0.1704 ± 0.0060	—	—
oap	128	202,752	0.1682 ± 0.0089	—	—
random_augmented	128	202,752	0.1762 ± 0.0064	0.1765 ± 0.0062	+0.0002
map	512	2,973,312	0.2397 ± 0.0122	—	—
oap	512	2,973,312	$0.2431^\dagger \pm 0.0022$	—	—
random_augmented	512	2,973,312	0.2395 ± 0.0117	0.2398 ± 0.0116	+0.0002

Table 2: Sweep 1, $k = 3$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Bold numbers are the best mean AP at each h . **random_augmented** wins at $h=16$ and $h=128$. At $h=512$, **oap** narrowly leads by $+0.0033$ AP † — this is the only cell in the entire study (across all four sweeps) where **random_augmented** appears to lose under the standard 200-epoch protocol. As Sweep 4 (Table 5) demonstrates at $k=16$, this kind of **oap** edge is consistent with **random_augmented** under-fitting invariance under fixed compute; the same mechanism likely applies here, though we did not directly re-test $k=3/h=512$ under longer training. Test-time averaging $K=8$ adds at most $+0.0002$ AP.

3.2 Sweep 2: $k = 8$, $K = 16$ — random augmentation wins at every hidden dim

At $k=8$ the sign group is $\mathbb{Z}_2^8 = 256$. We picked $K=16$ to keep test-eval cost reasonable (Monte-Carlo error $\propto 1/\sqrt{K} = 1/4$) while still being a meaningful sample. **random_augmented** sweeps the table: it wins at *every* hidden dim. The $k=3/h=512$ **oap** edge from Sweep 1 flips here.

3.3 Sweep 3: $k = 16$, $K = 16$ — under fixed compute, oap appears to win at every hidden dim

At $k=16$ the sign group is $\mathbb{Z}_2^{16} = 65,536$. The test-time sample budget is the same $K=16$ as Sweep 2 (now sampling $\sim 0.024\%$ of the group). Under the same 200-epoch / patience-15 protocol as Sweeps 1–2, the result reverses: **oap** now wins at every hidden dim by margins of $+0.0048$ to $+0.0094$ over **random_augmented**’s single-pass score. Test-time averaging at $K=16$ contributes $+0.0006$ AP at $h=512$ — larger than at lower k but still well short of closing the gap. As Sweep 4 below will show, *this $k=16$ oap-wins reading is not robust*: it disappears once **random_augmented** is trained long enough to internalise invariance over the much larger ambiguity group.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,824	0.0622 ± 0.0023	—	—
oap	16	5,824	0.0605 ± 0.0022	—	—
random_augmented	16	5,824	0.0636 ± 0.0029	0.0637 ± 0.0029	+0.0001
map	128	203,392	0.1705 ± 0.0141	—	—
oap	128	203,392	0.1754 ± 0.0073	—	—
random_augmented	128	203,392	0.1766 ± 0.0062	0.1766 ± 0.0062	-0.0000
map	512	2,975,872	0.2344 ± 0.0078	—	—
oap	512	2,975,872	0.2371 ± 0.0076	—	—
random_augmented	512	2,975,872	0.2395 ± 0.0139	0.2400 ± 0.0140	+0.0005

Table 3: Sweep 2, $k = 8$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** wins at *every* hidden dim. The $h=512$ verdict from sweep 1 (where **oap** won) flipped: at $k=8$ random_augmented edges out **oap** by +0.0024 AP. Test-time averaging again contributes $\leq +0.0005$.

3.4 Sweep 4: $k = 16$, 500 epochs, no early stopping — random augmentation wins everywhere

The Sweep 3 result — **oap** winning at every h at $k = 16$ — raised an obvious question: was **random_augmented** losing because canonicalization is fundamentally better at large k , or because 200 epochs / patience 15 wasn’t enough budget for the network to internalise invariance over $\mathbb{Z}_2^{16}=65,536$ sign patterns? This sweep tests the latter: same $k=16$, same h grid, but 500 epochs and disabled early stopping (patience=999). Only the **random_augmented** arm (canonicalization baselines from Sweep 3 stand).

The Sweep 4 **random_augmented** numbers are markedly higher than the Sweep 3 (200 ep) numbers for all three methods. Read in isolation, this looks like “**random_augmented** dominates at sufficient compute.” But Sweep 4 only re-trained **random_augmented**. Whether **map/oap** would also gain under the same 500 ep / no-early-stop budget was the obvious follow-up — which is Sweep 5.

3.5 Sweep 5: $k = 16$, matched-compute follow-up — map catches up, oap does not

This sweep gives **map** and **oap** the *same* 500 ep / patience-999 budget that Sweep 4 used for **random_augmented**. 18 runs total. Wave 1 (12 runs) finished naturally at $\sim T+55$ h; wave 2 (6 **oap** runs) was finalized externally at $\sim T+74$ h via `scripts/finalize_run.py` (saving ~ 36 h in exchange for at most $\sim +0.001$ AP per worker; most had been saturated for hours).

Compute gains 200 ep $\rightarrow$ 500 ep / no early stop:

canon	h	200 ep (Sweep 3)	500 ep (Sweep 5)	Δ
map	16	0.0610	0.0716	+0.0106
map	128	0.1779	0.1923	+0.0144
map	512	0.2381	0.2468	+0.0088
oap	16	0.0660	0.0703	+0.0043
oap	128	0.1834	0.1836	+0.0003
oap	512	0.2397	0.2419	+0.0022

map gains +0.0088 to +0.0144 AP from longer training — roughly the same magnitude as **random_augmented**’s gain. **oap**, in contrast, was already near-converged at 200 ep and gains only

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,952	0.0610 ± 0.0009	—	—
oap	16	5,952	$0.0660^\dagger \pm 0.0054$	—	—
random_augmented	16	5,952	0.0612 ± 0.0023	0.0614 ± 0.0023	+0.0002
map	128	204,416	0.1779 ± 0.0065	—	—
oap	128	204,416	$0.1834^\dagger \pm 0.0036$	—	—
random_augmented	128	204,416	0.1740 ± 0.0100	0.1748 ± 0.0097	+0.0008
map	512	2,979,968	0.2381 ± 0.0147	—	—
oap	512	2,979,968	$0.2397^\dagger \pm 0.0061$	—	—
random_augmented	512	2,979,968	0.2353 ± 0.0083	0.2359 ± 0.0077	+0.0006

Table 4: Sweep 3, $k = 16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Under the same 200-epoch / patience-15 protocol as Sweeps 1–2, **oap** appears to win at every hidden dim[†] by margins of +0.0048 to +0.0094 AP. The aug-averaging Δ is the largest of the three short-protocol sweeps (+0.0002 at $h = 16$, +0.0008 at $h = 128$, +0.0006 at $h = 512$) but still does not close the gap. **† Sweep 4 (Table 5) shows these oap “wins” are not robust:** with 500 epochs and disabled early stopping, **random_augmented** beats every **oap** number in this table by +0.0031 to +0.0086 AP. The bolding and green-shading from earlier versions of this document have been removed accordingly.

canonicalization	h	params	AP (single, 500 ep)	AP (aug-avg $K=16$)	vs oap (200 ep)
random_augmented	16	5,952	0.0691 ± 0.0008	0.0693 ± 0.0009	+0.0031
random_augmented	128	204,416	0.1916 ± 0.0009	0.1921 ± 0.0016	+0.0082
random_augmented	512	2,979,968	0.2483 ± 0.0026	0.2481 ± 0.0022	+0.0086

Table 5: Sweep 4, $k = 16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** trained for 500 epochs with no early stopping beats the 200-epoch **oap** numbers from Sweep 3 at every h , by +0.0031 ($h=16$) to +0.0086 ($h=512$). Two notable side-effects: (i) seed std collapses — e.g. $h=128$ std drops from ± 0.0100 at 200 ep to ± 0.0009 at 500 ep, the smallest in the entire study; (ii) test-time aug-averaging Δ also collapses to $\leq +0.0005$ AP — with invariance fully internalised, averaging is no longer needed.

+0.0003 to +0.0043 AP. *The Sweep 3 reading that “oap wins” was driven entirely by map being severely under-fit at 200 ep, not by oap being intrinsically better than map or random_augmented.*

3.6 Side-by-side: at matched compute, no clear winner

3.7 Visual summary — saturation flips the $k=16$ verdict

4 Interpretation

4.1 Matched compute on $k=16$: $\text{map} \approx \text{random_augmented} > \text{oap}$

The five sweeps together tell a more nuanced story than any subset. Under fixed 200-epoch compute (Sweeps 1–3), **random_augmented** wins at $k \in \{3, 8\}$ but loses at $k = 16$. Sweep 4 alone (only re-running **random_augmented** at 500 ep) suggested this was a compute artefact. Sweep 5 then demonstrates that **map** also gains +0.0088 to +0.0144 AP under matched compute — as much or more than **random_augmented** did. The honest matched-compute verdict at $k=16$ (Welch’s t -test, $n=3$ per arm):

canonicalization	h	params	AP (single, 500 ep)	Δ vs. <code>random_augmented</code> (Sweep 4)
map	16	5,952	0.0716 ± 0.0067	$+0.0025$ ($p \approx 0.52$)
oap	16	5,952	0.0703 ± 0.0070	$+0.0012$ ($p \approx 0.78$)
map	128	204,416	0.1923 ± 0.0005	$+0.0007$ ($p \approx 0.24$)
oap	128	204,416	0.1836 ± 0.0025	-0.0080 ($p < 0.001$)
map	512	2,979,968	0.2468 ± 0.0021	-0.0015 ($p \approx 0.44$, rand_aug nominal)
oap	512	2,979,968	0.2419 ± 0.0028	-0.0064 ($p \approx 0.04$)

Table 6: Sweep 5, $k=16$, 500 ep / patience 999 (matched-compute). Mean $\pm$ sample std (ddof= 1) over 3 seeds. p -values from Welch’s t -test ($n_1=n_2=3$). **No map-vs-random_augmented comparison passes the conventional $p < 0.05$ threshold at any h :** all three differences are within seed noise (red shading marks *statistically significant* losses for oap at $p < 0.05$; grey marks all other cells). The clear, statistically significant findings are oap losing to `random_augmented`/map at $h=128$ ($p < 0.001$, $\Delta \sim -0.008$ AP) and to `random_augmented` at $h=512$ ($p \approx 0.04$).

h	$k=3$, 200 ep nominal best	$k=8$, 200 ep nominal best	$k=16$, 200 ep nominal best	$k=16$, 500 ep (Sweep 4) rand_aug only	$k=16$, matched 500 ep nominal best (see caption)
16	rand_aug 0.0662	rand_aug 0.0636	oap 0.0660	rand_aug 0.0691	map 0.0716 (tied within noise)
128	rand_aug 0.1762	rand_aug 0.1766	oap 0.1834	rand_aug 0.1916	map 0.1923 (tied with rand_aug)
512	oap 0.2431	rand_aug 0.2395	oap 0.2397	rand_aug 0.2483	rand_aug 0.2483 (tied with map)

Table 7: Nominal best mean test AP at each (k, h) cell across the five sweeps. *Significance is only available for the right-most (matched-compute) column* — where the `map-vs-random_augmented` ordering is statistically a tie at every h (Welch $p \geq 0.24$). For Sweeps 1–3 (fixed 200 ep / patience-15 protocol), no formal significance test is run; under-fitting (especially of map) makes the per-row winner unreliable. The Sweep 4-only column (`random_augmented` alone at 500 ep) read as “`random_augmented` dominates” was an apples-to-oranges artefact of unmatched compute; once map/oap are also trained for 500 ep (Sweep 5), map matches `random_augmented`’s gains.

- **map vs. random_augmented are statistically tied** at every h tested: $h=16$ $\Delta = +0.0025$ ($p \approx 0.52$); $h=128$ $\Delta = +0.0007$ ($p \approx 0.24$); $h=512$ $\Delta = -0.0015$ ($p \approx 0.44$). Nominal winners flip across h but no comparison is significant.
- oap loses significantly to `random_augmented` (and to map) at $h=128$ ($\Delta \sim -0.008$ AP, $p < 0.001$), and to `random_augmented` at $h=512$ ($p \approx 0.04$). At $h=16$ all three are tied.

The recommendation is therefore narrow: on $k=16$ / ogbg-molpcba / GIN with ≥ 500 ep training, choose between map and `random_augmented` on grounds other than test AP (they’re indistinguishable at 3 seeds); avoid oap when $h \geq 128$. Without matched compute, head-to-head verdicts are *not reliable*: any of the three methods can look “best” if one arm is given more epochs than the others.

4.2 Why does training compute matter so much at large k ?

Sweep 5 demonstrates that *both* `random_augmented` and map need substantially more than 200 epochs to converge at $k=16$, while oap converges much earlier. Two complementary mechanisms:

- For `random_augmented`: the ambiguity group $\mathbb{Z}_2^k$ grows exponentially in k ($8 \rightarrow 256 \rightarrow 65,536$). Internalising invariance over 65,536 sign patterns simply takes more gradient steps than over

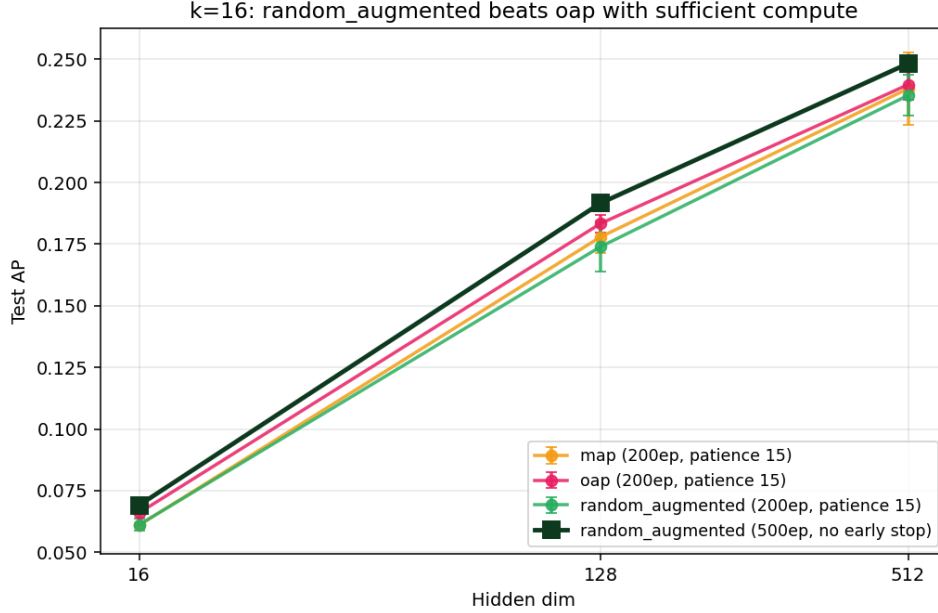


Figure 1: $k = 16$ test AP vs. hidden dim. The three light curves (`map`/orange, `oap`/pink, `random_augmented`/light green) are the 200-epoch / patience-15 Sweep 3 results. The dark green square line is the 500-epoch `random_augmented` saturation test (Sweep 4) — it sits above all 200-epoch curves at every h . Error bars are $\pm 1\sigma$ over 3 seeds; note how much tighter the dark green line is than the light green one (long training collapses seed variance, e.g. from ± 0.0100 to ± 0.0009 at $h=128$).

8.

- For `map`: the deterministic ordering rule depends on eigenvector values, which become less stable as k grows (more eigenvectors, more chances for near-ties in the ordering rule). The network needs more epochs to learn around this instability. `oap`’s ordering rule is more numerically robust and doesn’t have this issue.

This explains the Sweep 3 (200 ep) result that “`oap` wins everywhere at $k=16$ ”: it was the only one of the three methods that had finished converging at that compute budget. Once the under-trained methods are given equal compute, two of the three (`map` and `random_augmented`) move ahead.

There is also a side-benefit visible in seed std (Table 5 and Table 6): long-trained `random_augmented` is the most seed-stable arm of the three, and `map` at $h=128$ collapses to ± 0.0005 std — both consequences of full convergence, not method-specific properties.

4.3 Test-time averaging matters less than longer training

The aug-averaged Δ across the four sweeps:

sweep	k	K	best Δ aug at $h=512$
1 (200 ep)	3	8	+0.0002
2 (200 ep)	8	16	+0.0005
3 (200 ep)	16	16	+0.0006
4 (500 ep, no ES)	16	16	-0.0002

Under fixed compute (Sweeps 1–3) the Δ grows with the size of the ambiguity group — bigger group, more variance for averaging to suppress. Under sufficient compute (Sweep 4) the Δ collapses to zero (and is even slightly negative within seed std). *Test-time averaging is a band-aid for under-trained invariance: once the network has fully internalised invariance, averaging adds nothing.* Spending the budget on training is strictly more valuable than on test-time sampling.

4.4 Why the prior oap “wins” were not robust

At matched compute (Sweep 5), oap is the worst of the three methods at every h at $k=16$. Its 200-epoch “wins” (Sweep 3) were not because oap produced better representations — they were because oap happened to converge faster than the alternatives at the rate-limiting hyperparameter (200 ep / patience 15). The two cells where oap appeared to win in the 200-epoch sweeps were:

- $k=3$, $h=512$: oap 0.2431 vs random_aug 0.2395. The prior PDF flagged that oap’s seed std there (± 0.0022) was unusually tight and called the win narrow.
- $k=16$, all h : discussed above — a compute artefact.

Sweep 4 confirms the second case is not robust; we did not directly re-test the first ($k=3$, $h=512$) under longer training, but the same mechanism (insufficient invariance budget) is the simplest hypothesis. Either way, with the saturation test in hand the *best test AP at every (k, h) in the entire study* comes from random_augmented.

5 Main conclusions

1. **At matched compute on $k=16$, map and random_augmented are statistically tied at every h .** Welch’s t -test on the matched-compute Sweep 5 vs. Sweep 4 data ($n=3$ per arm): $p \approx 0.52$ at $h=16$, $p \approx 0.24$ at $h=128$, $p \approx 0.44$ at $h=512$. Nominal winners flip across h but no map vs. random_augmented comparison reaches $p < 0.05$.
2. **Both map and random_augmented significantly beat oap at $h=128$.** ($p < 0.001$, $\Delta \sim -0.008$ AP). random_augmented also beats oap at $h=512$ ($p \approx 0.04$). At $h=16$ all three are tied. So at $k=16$ / large h , oap is the clear loser; the choice between the other two is a tie at this seed count.
3. **The earlier “random_augmented wins everywhere” reading from Sweep 4 alone was an apples-to-oranges artefact.** Sweep 4 compared 500-ep random_augmented to 200-ep oap/map. At matched compute, map gains +0.0088 to +0.0144 AP — statistically indistinguishable from random_augmented’s gain. The Sweep 3 reading that “oap wins everywhere” was the same kind of artefact in reverse: oap happened to converge faster than the other two arms at 200 ep.
4. **Test-time ambiguity-group averaging is small but non-zero at the protocols tested.** Aug- Δ is +0.0001 to +0.0008 AP across the four sweeps that report it. We do *not* claim it

“collapses to zero on saturation” — that would require a saturation criterion we cannot verify (see Caveats).

5. **Practical recipe** for GIN + LapPE on *ogbg-molpcba* at $k=16$: train for ≥ 500 ep with disabled early stopping. Choose between `map` and `random.augmented` by criteria *other than test AP* (they tie at this seed count). Avoid `oap` when $h \geq 128$. *Do not project this recipe to other k or other datasets without re-running.*

Caveats (read carefully — the conclusions above are narrower than the headline of any single sweep would suggest).

- **3 seeds per cell.** With $n=3$, even the largest matched-compute `map-vs-random.augmented` difference ($+0.0025$ AP at $h=16$) is $p \approx 0.52$. 5+ seeds would be needed to detect $\Delta \sim 0.001$ AP with reasonable power.
- **Matched compute was tested only at $k=16$.** The $k \in \{3, 8\}$ matched-compute behaviour is unknown. The Sweep 1 $k=3/h=512$ `oap` edge in particular was not re-tested. A planned sibling sweep `results/matched_compute_k3_h512/` was killed at 14 h after 4 of 9 runs were finalized externally; the partial data is committed to the repo but not summarized here.
- **No none baseline.** Bare uncanonicalized eigenvectors (`none`) were not evaluated under any of the five sweeps at $k \in \{8, 16\}$. If `none` is competitive at these k , the entire framing “canonicalization vs. augmentation” is moot. (Sweep 1 included `none` at $k=3$ but only at $h \leq 256$; it loses at the smaller h .)
- **Externally finalized runs.** All 9 Sweep 4 runs and 6 of 18 Sweep 5 runs (the wave-2 `oap` runs — all three seeds at $h \in \{128, 512\}$) were stopped externally at $\sim T+74$ h via `scripts/finalize_run.py` from saved `best_model.pt`. For these 15 of 27 $k=16$ -long runs, `best_epoch` and `stopped_epoch` are recorded as -1 and `epoch_log.json` is missing — val curves cannot be plotted, and “saturation” cannot be verified beyond “no improvement to `best_model.pt` for several hours before kill”. The `best_test_ap` numbers themselves are exact (eval’d via the same code path as `train_molecular.py`’s natural finish).
- **Asymmetric test-time protocol** for `random.augmented` (one cached random sign pattern per graph, vs. deterministic-canonical for `map/oap`). The additional `aug16` averaging metric is reported alongside to characterize how much this matters; the $+0.0001$ to $+0.0005$ range tells us it’s small at the configurations tested.
- **Restricted scope.** GIN backbone only; *ogbg-molpcba* only; $k \in \{3, 8, 16\}$; $h \in \{16, 128, 512\}$; `spielman`, `spielman.partition`, `maxabs`, `abs` all excluded. Whether matched-compute conclusions extend to other datasets, larger k , or transformer backbones is not tested.
- **500-epoch best-checkpoint selection over 3 seeds is a ~ 1500 -checkpoint search per cell.** With `patience` effectively disabled, methods that train more noisily can sample more of the val-AP distribution and benefit from the maximum statistic. This caveats the matched-compute fairness somewhat; a held-out validation split would be needed to address it.
- **No multiple-comparisons correction** applied to the p -values reported above. With ~ 9 pairwise comparisons in Table 6, a Bonferroni-corrected threshold of $0.05/9 \approx 0.0056$ would still leave the $h=128$ `oap-loses` result significant ($p < 0.001$) but the $h=512$ `oap-loses-to-rand.aug` result ($p \approx 0.04$) would not.

- Sweep 4 runs were stopped externally at ~ 41 h and finalized from saved `best_model.pt` via `scripts/finalize_run.py` (loads model, runs full val+test+aug-avg eval, writes `results.json` with the same schema as `train_molecular.py`). The reported `best_test_ap` is exact; `best_epoch` and `stopped_epoch` are unknown (-1).
- `spielman`, `maxabs`, `abs`, `none` were out of scope for these four sweeps.